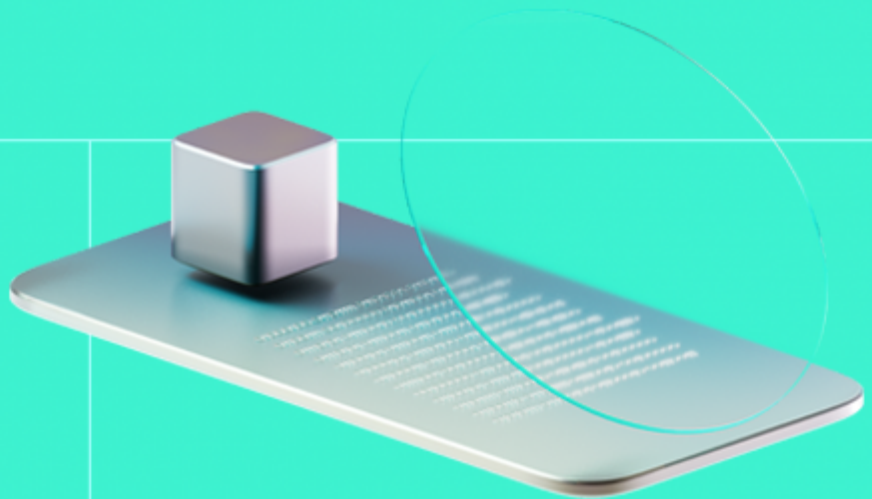




Smart Contract Code Review And Security Analysis Report

Customer: Seedify.fund

Date: 29/07/2025



We express our gratitude to the Seedify.fund team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Seedify is a bonding curve system that has a decentralized token launch mechanism that implements a mathematical pricing curve for token sales.

Document

Name	Smart Contract Code Review and Security Analysis Report for Seedify.fund
Audited By	Turgay Arda Usman, Khrystyna Tkachuk
Approved By	Ataberk Yavuzer
Website	https://launchpad.seedify.fund/
Changelog	16/07/2025 - Preliminary Report 29/07/2025 - Final Report
Platform	Ethereum, BNB, Avalanche
Language	Solidity
Tags	Launchpad, Vesting
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/Seedifyfund/seedify-era-contracts
Initial Commit	7b054d9
Final Commit	1761ce3

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

18

Total Findings

17

Resolved

1

Accepted

0

Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	5
Low	2

Vulnerability	Severity	Status
F-2025-11617 - Users Can Perform New Tokens Purchase After autoRefund() Leading to Underflow in claim()	High	Fixed
F-2025-11651 - Precision Loss in Bonding Curve Calculations	High	Fixed
F-2025-11654 - Flawed Rounding Logic in calculateBuyAmount Leads to Loss of Funds	Medium	Fixed
F-2025-11735 - Incorrect Progressive-TGE Calculation Order Leads to Unintended Token Unlocks	Medium	Fixed
F-2025-11742 - Incorrect Handling of reserveFee During dex() Execution	Medium	Fixed
F-2025-11752 - Incorrect Fee Math Allows Users to Exceed Allocation	Medium	Fixed
F-2025-11767 - Inconsistent State Change in autoRefund() Affects TGE Unlock and Price Logic	Medium	Fixed
F-2025-11734 - Missing Validation on Critical Vesting Parameters	Low	Fixed
F-2025-11748 - Incorrect Calculation Can Bypass Vesting Schedule	Low	Accepted
F-2025-11655 - Floating Pragma	Info	Fixed
F-2025-11743 - Unused Custom Errors and Events	Info	Fixed
F-2025-11744 - Typographical Error in BondingCurveTypes	Info	Fixed
F-2025-11749 - Misleading 'start' Field in BondingCurveTimestamps Struct	Info	Fixed
F-2025-11750 - Redundant SafeERC20 and IERC20 Imports Declaration	Info	Fixed

Vulnerability	Severity	Status
F-2025-11751 - State Change of _maxLoop Without Event in BondingCurveFactory Contract	Info	Fixed
F-2025-11759 - Redundant OFTTypes and BondingCurveStorage Import Declaration	Info	Fixed
F-2025-11765 - Unsafe Salt Generation with abi.encodePacked	Info	Fixed
F-2025-11948 - Code Contains `hardhat` Import	Info	Fixed

Documentation quality

- Functional requirements are provided.
- Technical description is provided.

Code quality

- The code mostly follows best practices and style guides:
 - For more information please advise informational findings
- The development environment is configured.

Test coverage

Code coverage of the project is **38.52%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is missed.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview	7
Privileged Roles	7
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	58
Appendix 1. Definitions	59
Severities	59
Potential Risks	59
Appendix 2. Scope	60
Appendix 3. Additional Valuables	61

System Overview

Seedify is a bonding curve system that has a decentralized token launch mechanism that implements a mathematical pricing curve for token sales. It has the following contracts:

BondingCurveFactory — a factory contract responsible for creating and managing bonding curve instances.

BondingCurveLibrary — contract implements an exponential bonding curve mechanism for token pricing and trading.

BondingCurve — the primary entry point that inherits from both readable and writable components.

BondingCurveTypes — contains all structs, enums, events, and errors used throughout the system.

BondingCurveStorage — implements diamond storage pattern for upgradeable contracts.

BondingCurveReadable — provides view functions for contract state queries.

BondingCurveWritable — handles all state-changing operations.

BondingCurveWritableRestricted — contains admin-only operations.

Privileged roles

- The admin can initialize a contract with validation.
- The admin can pause or unpause the system.
- The admin can add liquidity to DEX upon completion.
- The admin can withdraw LP tokens.
- The admin can perform an emergency token withdrawal.

Potential Risks

- The functioning of the system significantly relies on specific external contracts. Any flaws or vulnerabilities in these contracts adversely affect the audited project, potentially leading to security breaches or loss of funds.
- The implemented liquidity addition logic highly depends on external contracts not covered by the audit. This reliance introduces risks if these external contracts are compromised or contain vulnerabilities, affecting the audited project's integrity.
- The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.

Findings

Vulnerability Details

[F-2025-11617](#) - Users Can Perform New Tokens Purchase After autoRefund() Leading to Underflow in claim() - High

Description:

The `BondingCurve` contract enables users to purchase project tokens using a bonding mechanism and later claim them once the vesting period begins. The `BondingCurveWritable` extension allows the default admin to initiate an automatic refund of the bonded token via the `autoRefund()` function if the soft cap is not reached.

However, during `autoRefund()`, the user's state is not properly updated to reflect the refund. Specifically, the `user.isRefunded` flag is not set to true, and no adjustments are made to the `amountToClaim` or `amountClaimed` fields. This allows users to buy tokens again in the same bonding curve after being automatically refunded. But because `amountClaimed` is set to `type(uint256).max` during the refund process, it becomes impossible for that user to claim any newly purchased tokens, leading to a loss of bonded tokens and claimable project token amount.

BondingCurveWritable::autoRefund():

```
function autoRefund(
    address[] calldata userAddresses
) external whenNotPaused onlyRole(DEFAULT_ADMIN_ROLE) {
    if (userAddresses.length == 0) revert EmptyArray();

    BondingCurveStorage.Storage storage strg = BondingCurveStorage.layout
();
    if (strg.ledger.raisedTokenAmount >= strg.setup.softCap)
        revert RefundNotAllowed();

    address token = strg.setup.bondedTokenDetails.token;
    uint256 totalBondedAmount;
    uint256 totalClaimAmount;

    for (uint256 i; i < userAddresses.length; i++) {
        User storage user = strg.user[userAddresses[i]];
        uint256 bondedAmount = user.bondedTokenAmount;
        if (bondedAmount > 0) {
            totalBondedAmount += bondedAmount;
```

```

        totalClaimAmount += user.amountToClaim;

        user.bondedTokenAmount = 0;
        user.amountClaimed = type(uint256).max;

        _giveToken(token, userAddresses[i], bondedAmount);
        emit Refunded(
            userAddresses[i],
            strg.ledger.raisedTokenAmount,
            getAmountOutForBondedToken(1 ether)
        );
    }
}

strg.ledger.tokenToSell += totalClaimAmount;
strg.ledger.raisedTokenAmount -= totalBondedAmount;
}

```

Refunded users can re-purchase tokens due to missing `isRefunded` flag, but will be permanently blocked from claiming them because underflow occurs once claimable amount is calculated.

BondingCurveWritableRestricted::_getClaimableAmount():

```

function _getClaimableAmount(
    User memory user,
    VestingDetails memory vestingDetails
) internal view returns (uint256 claimableAmount) {
    ...
    if (block.timestamp < vestingDetails.endDate) {
        claimableAmount = _claimableAmount(
            vestingDetails,
            amount,
            startAmount,
            1e36
        );
    } else {
        claimableAmount = amount;
    }

    claimableAmount -= user.amountClaimed; // <-- underflow
}

```

By contrast, the `refund()` function correctly sets the `user.isRefunded = true`, and this status is enforced in `_buy()` to prevent re-buying.

Furthermore, if the user buys again after being auto refunded, the claimable token amount may include previously refunded amounts since `user.amountToClaim` field is not updated in the refund function. This

could cause discrepancies and incorrect accounting, particularly when admins later attempt to refund users via the `withdraw()` function that serves as emergency withdrawal mechanism.

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: Consider setting `user.isRefunded = true` in the `autoRefund()` function, as is done in `refund()` to prevent refunded user from re-buying in the same curve. If re-buying after an admin auto refund is expected behavior, ensure storage variables are correctly re-initialized to reflect a fresh state.

Resolution: Fixed in `90b3457`: added flag `user.isRefunded = true` to the `autoRefund()` function that prevents users from purchasing again on a refunded curve.

Evidences

PoC

Reproduce:

```
it("User can't claim their tokens after autoRefund is perform", async function () {
  const { accounts, setUp, bondingCurve, paymentToken, getProof, bondingCurveLibrary, totalSupply } =
    await loadFixture(BondingCurveSetupFixture);
  await time.increaseTo(setUp.bondingCurveTimestamps.tier9);
```

```

const proof = (await getProof(accounts.user1.address)) as `0x${string}`[];
let amountinExpected = await bondingCurve.read.getAmounOutForProjectToken([
parseEther("500000000")]);
// Buy tokens th first time
await bondingCurve.write.buy(
[
{
user: accounts.user1.address,
amount: parseEther("500000000"),
roundType: 0,
},
proof,
amountinExpected,
0n,
],
{ account: accounts.user1 }
);

console.log("Account 1:", await bondingCurve.read.getUser([accounts.user1.a
ddress]))

console.log("Auto refund performing...")
// Admin permorms auto refund: revert user allocation and return bonded tok
ens
await bondingCurve.write.autoRefund([[accounts.user1.address]], {
account: accounts.platformOwner,
});

console.log("Account 1 after auto refund: \n", await bondingCurve.read.getU
ser([accounts.user1.address]))

amountinExpected = await bondingCurve.read.getAmounOutForProjectToken([pars
eEther("500000000")]);
// The same autoRefund account can buy more tokens
await bondingCurve.write.buy(
[
{
user: accounts.user1.address,
amount: parseEther("500000000"),
roundType: 0,
},
proof,
amountinExpected,
0n,
],
{ account: accounts.user1 }
);

```

```

    amountinExpected = await bondingCurve.read.getAmountOutForProjectToken([parseEther("5000000000")]);

    const proof2 = (await getProof(accounts.user2.address)) as `0x${string}`[];

    // Performing more buy to reach softCap
    await bondingCurve.write.buy(
      [
        {
          user: accounts.user2.address,
          amount: parseEther("5000000000"),
          roundType: 0,
        },
        proof2,
        amountinExpected,
        0n,
      ],
      { account: accounts.user2 }
    );

    console.log("Account 1: amountClaimed >=: amountToClaim\n", await bondingCurve.read.getUser([accounts.user1.address]));
    await time.increaseTo(Number(setUp.bondingCurveTimestamps.end));
    // Performing dex function to start vesting period
    await bondingCurve.write.dex({ account: accounts.platformOwner });
    await time.increaseTo(Number(setUp.vestingDetails.startDate));
    // Refunded account trying to claim but fails due to amountClaimed was set to uint256.max in autoRefund operation
    await bondingCurve.write.claim({ account: accounts.user1 }); // reverted due to amountClaimed is uint256.max - underflow
  });

```

Results:

```

BondingCurve
  Refund
ending Price: 2000000000000000000n
totalSupply: 5800000000000000000000000000000000n
ending Price: 2000000000000000000n
HARD CAP: 836763123715598776184282919n
hardCapIncludeTaxAmount: 843086270746195240487942487n
Account 1: {
  amountToClaim: 3194948990000000000000000000000n,
  amountClaimed: 0n,
  bondedTokenAmount: 389054584920000000000000000000n,
  tgeAmount: 2046684322994000000000000000000n,
  isRefunded: false
}
Auto refund performing...
Account 1 after auto refund:

```

```

{
  amountToClaim: 3194948990000000000000000000n,
  amountClaimed: 115792089237316195423570985008687907853269984665640564039457
584007913129639935n,
  bondedTokenAmount: 0n,
  tgeAmount: 2046684322994000000000000000n,
  isRefunded: false
}
Account 1: amountClaimed >=: amountToClaim
{
  amountToClaim: 5607262390000000000000000000n,
  amountClaimed: 115792089237316195423570985008687907853269984665640564039457
584007913129639935n,
  bondedTokenAmount: 3391163449725000000000000000n,
  tgeAmount: 3732167695574000000000000000n,
  isRefunded: false
}

```

1) User can't claim their tokens after autoRefund is perform

0 passing (553ms)

1 failing

1) BondingCurve

Refund

User can't claim their tokens after autoRefund is perform:

ContractFunctionExecutionError: An unknown RPC error occurred.

Request Arguments:

```

from: 0x14dc79964da2c08b23698b3d3cc7ca32193d9955
to:    0x0165878a594ca255338adfa4d48449f69242eb8f
data:  0x4e71d92d

```

Contract Call:

```

address: 0x0165878a594ca255338adfa4d48449f69242eb8f
function: claim()
sender:   0x14dc79964da2c08b23698b3d3cc7ca32193d9955

```

Docs: <https://viem.sh/docs/contract/writeContract>

Details: VM Exception while processing transaction: reverted with panic code 0x11 (Arithmetic operation overflowed outside of an unchecked block)

Version: viem@2.23.2

at getContractError (node_modules/viem/utils/errors/getContractError.ts:78:10)

at writeContract (node_modules/viem/actions/wallet/writeContract.ts:208:27)

at async Context.<anonymous> (test/unit/bondingcurve/refundAudit.test.t

s:233:7)

Caused by: TransactionExecutionError: An unknown RPC error occurred.

[F-2025-11651](#) - Precision Loss in Bonding Curve Calculations - High

Description:

The `calculateBuyCost` function is designed to calculate the cost of purchasing a specific amount of tokens from an exponential bonding curve. The pricing should be continuous, meaning the cost is smoothly influenced by the exact number of tokens already sold, including fractional amounts (represented in `wei`).

The core mathematical formula discards all fractional token precision by performing integer division on the `soldSoFarWei` and `sellableSupply` amounts before converting them to fixed-point numbers. This fundamentally changes the behavior of the continuous exponential curve into a discrete step-function where the price only updates when a full integer token has been sold. This leads to inaccurate pricing, causing users to consistently overpay or underpay and creating a growing mismatch between the funds held by the contract and the value predicted by its own formula.

The precision loss occurs in the `_area0` private function. Before performing high-precision fixed-point math, the number of tokens sold (`sWei`) is truncated to an integer.

```
function _area0(  
    uint256 sWei,  
    // ...  
) private pure returns (uint256) {  
    // number of tokens (human)  
    uint256 supplyTokens = sellableSupply / SCALE;  
    int128 fracQ64 = (sWei / SCALE).fromUInt().div(supplyTokens.fromUInt());  
    // ...  
}
```

The expression `sWei / SCALE` converts a wei value (e.g., `10.5 * 1e18`) into an integer token amount (`10`). Any fractional part is completely discarded. As a result, the calculated cost to buy tokens is identical whether `10.01` tokens or `10.99` tokens have been sold. The price only changes when the `soldSoFarWei` amount crosses a full token boundary (e.g., from `10.99` to `11.00`). This violates the mathematical principles of a smooth bonding curve.

This can lead to:

- Users will not receive a fair market price based on the exact state of the curve. They will either be overcharged or

undercharged depending on where their purchase falls within the "step."

- Over time, the accumulated inaccuracies will cause the total funds held in the contract to diverge from the theoretical value that should be held according to the bonding curve integral. This can lead to a slow but steady drain of value from the protocol or, conversely, from its users.

Assets:

- BondingCurveLibrary.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation: To preserve precision, the division should be performed using the full wei values with the `ABDKMath64x64` library, which is designed for this purpose. Modify the calculation in the `_area0` function to use `divu`, which performs division on unsigned integers and returns a high-precision fixed-point number.

Resolution: In commit `f6e452b`, the `ABDKMath64x64` library is utilized.

Evidences

PoC

Reproduce:

1. The test first calls `calculateBuyCost` pretending exactly `10.00` tokens have been sold. This gives us a baseline cost, which we'll call `costInteger`.
2. It immediately calls `calculateBuyCost` again with the same parameters, but this time pretending `10.25` tokens have been

- sold. A mathematically correct bonding curve should produce a slightly higher cost. We'll call this result `costFractional`.
3. Inside the smart contract, the `soldSoFarWei` value is divided by `1e18` using integer math before any other calculations.
 1. $10.00 * 1e18 / 1e18$ results in 10.
 2. $10.25 * 1e18 / 1e18$ also results in 1
 4. The test asserts that `costFractional` is **exactly equal to** `costInteger`.
 5. Finally, the test calculates a third cost at the `11.00` token mark (`costNextToken`).

PoC Code:

```
const { ethers } = require("hardhat");
import { expect } from "chai";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

describe("BondingCurveLibrary – Precision Loss", function () {
  async function deployLibraryFixture() {
    const factory = await ethers.getContractFactory("BondingCurveLibrary");
    const library = await factory.deploy();
    return { library };
  }

  it("ignores fractional tokens when computing buy cost (0.25 token step proves loss)", async function () {
    const { library } = await loadFixture(deployLibraryFixture);

    const amountWei = ethers.parseEther("1"); // purchasing 1 token

    const endingPrice = ethers.parseEther("100");
    const multiplierHuman = 200; // > 100 to avoid underflow/zero-div issues
    const sellableSupply = ethers.parseEther("1000");

    const soldInteger = ethers.parseEther("10");
    const soldFractional = ethers.parseEther("10.5");

    const costInteger = await library.calculateBuyCost(
      amountWei,
      soldInteger,
      endingPrice,
      multiplierHuman,
```

```

sellableSupply
);

const costFractional = await library.calculateBuyCost(
  amountWei,
  soldFractional,
  endingPrice,
  multiplierHuman,
  sellableSupply
);

expect(costFractional).to.equal(costInteger);

const soldNextToken = ethers.parseEther("11");
const costNextToken = await library.calculateBuyCost(
  amountWei,
  soldNextToken,
  endingPrice,
  multiplierHuman,
  sellableSupply
);

expect(costInteger).to.be.gt(0);
expect(costNextToken).to.be.gt(costInteger);
});
});

```

Results:

BondingCurveLibrary – Precision Loss

✓ ignores fractional tokens when computing buy cost (0.25 token step proves loss) (2677ms)

[F-2025-11654](#) - Flawed Rounding Logic in calculateBuyAmount Leads to Loss of Funds - Medium

Description:

The `calculateBuyAmount` function is the inverse of `calculateBuyCost`. It is intended to take a specific cost (in `wei`) and determine the exact amount of tokens that can be purchased. This function is critical for user-facing applications to query purchase amounts before submitting a transaction.

The function contains a "smart rounding" feature that aggressively rounds the calculated purchase amount to the nearest whole token. If a user's provided `cost` corresponds to a purchase of less than `0.5` tokens, the function will round the amount down to 0. This creates a situation where a user can pay a non-zero, tangible cost but receive zero tokens in return, resulting in a direct and irreversible loss of the user's funds.

The issue is located at the end of the `_inverseArea0` function, which is called by `calculateBuyAmount`. After correctly calculating the token amount `sWei` that corresponds to the `targetArea`, the code rounds it to the nearest multiple of `1e18` (`SCALE`).

```
function _inverseArea0(
    // ...
) private pure returns (uint256) {
    // ... preceding calculations ...
    uint256 sWei = fracQ64.mul(SCALE * supplyTokens);

    // Smart rounding: round to nearest wei to eliminate calculation drift
    // while preserving sensitivity to small cost changes
    uint256 remainder = sWei % SCALE;

    // Round to the nearest SCALE unit, but with high precision consideration
    if (remainder >= SCALE / 2) { // e.g. 0.5 * 1e18
        sWei = sWei + (SCALE - remainder);
    } else {
        sWei = sWei - remainder;
    }
    // ...
}
```

This logic creates a "dead zone" for purchases. If `calculateBuyAmount` is called with a cost that would result in buying, for example, `0.4` tokens, the logic rounds this down to `0`. The calling contract, however, would have already accepted the user's payment, leading

to theft. The comment incorrectly states it rounds to the "nearest wei" when it is actually rounding to the nearest full token.

This leads any transaction where a user intends to buy an amount of tokens between (0 , 0.5) to result in the user losing their entire payment for that transaction and receiving nothing.

Assets:

- BondingCurveLibrary.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: The rounding logic is flawed and dangerous, and it should be removed entirely. The precision offered by the fixed-point math library is sufficient to return an accurate fractional token amount.

Resolution: In commit [f6e452b](#), the rounding logic has been removed.

Evidences

PoC

Reproduce:

1. A user calls `buy()` or `buyPublic()` with a specific amount. The contract immediately takes their payment (minus fees) via the `_transferToken` call. At this point, the user's funds are gone.
2. The contract then calls the vulnerable `calculateBuyAmount` function to determine how many tokens the user should get.
3. If the user's amount corresponds to a purchase of < 0.5 tokens, `calculateBuyAmount` will round down and return `tokensToGive = 0`.
 1. Assume the `calculateBuyAmount` has 0.4 tokens

2. The library's internal logic correctly calculates that `0.1` tokens at this point on the curve should buy user exactly `0.4` PROJ tokens.
3. Since the calculated amount (`0.4` tokens) is less than the rounding threshold (`0.5` tokens), the logic rounds the amount down to zero.
4. The contract proceeds to update the user's state

```
user.amountToClaim += tokensToGive; // Adds 0
```

Results:

The user's `bondedTokenAmount` is increased, but their `claimable` token amount is not.

[F-2025-11735](#) - Incorrect Progressive-TGE Calculation Order Leads to Unintended Token Unlocks - Medium

Description:

The `buy` function facilitates token purchases and subsequently calculates the buyer's Token Generation Event (TGE) allocation using the `_calculateTGEPercent` internal function. The TGE percentage is designed to be progressive: it should start at a `baseTGEUnlockPercent` for the very first tokens sold and scale up towards `100%` for the very last tokens sold.

The contract updates the `totalProjectTokenSold` state variable before it calculates the progressive TGE percentage for the current buyer. This flawed order of operations has two major consequences. First, every buyer receives a slightly larger TGE allocation than intended. Second, and more critically, it allows for a scenario where a user who buys all remaining tokens in a single transaction will receive a `100%` TGE on their entire purchase, completely bypassing the vesting schedule for that amount. This undermines the core tokenomic model and creates a significant economic vulnerability.

In the `buy` function, the `totalProjectTokenSold` is incremented. Immediately after, this updated value is used to calculate the TGE percentage for the user's purchase.

```
...  
    strg.ledger.totalProjectTokenSold += tokensToGive;  
    uint256 tgePercent = _calculateTGEPercent(  
        strg.ledger.tokenToSell,  
        strg.ledger.totalProjectTokenSold, /  
        strg.setup.vestingDetails.baseTGEUnlockPercent  
    );  
    user.tgeAmount += (tokensToGive * tgePercent) / 10000;  
...
```

The `_calculateTGEPercent` function calculates a ratio of `totalProjectTokenSold` to `totalTokenToSell`.

```
uint256 ratio = (totalProjectTokenSold * 1e18) / totalTokenToSell;
```

Because the state is updated first, this ratio is always higher than it should be. The bug occurs if a user buys all remaining tokens. In this case, `totalProjectTokenSold` becomes equal to `totalTokenToSell` before the calculation. This makes the ratio equal to `1e18 (or 100%)`. The subsequent calculation becomes:

```
scaledUnlock = baseTGEUnlockPercent * 1e18 + 1e18 * (10000 -
baseTGEUnlockPercent)

scaledUnlock = 1e18 * (baseTGEUnlockPercent + 10000 -
baseTGEUnlockPercent)scaledUnlock = 1e18 * 10000
```

The function then returns `scaledUnlock / 1e18`, which is `10000 (100%)`. Therefore, the user who bought the last chunk of tokens gets an immediate `100%` TGE on their entire purchase, no matter how large it was. This is not the intended behavior; only the very last token should receive the `100%` TGE rate, not the entire final purchase.

A strategic user (or bot) can wait until the final moments of the sale to make a single large purchase, acquiring all remaining tokens. This action would grant them a `100%` immediate unlock on that entire amount, allowing them to bypass the vesting schedule that all previous, smaller buyers were subject to.

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: To ensure the TGE calculation is based on the state at the time of purchase and that the progressive model works as intended, the logic must be re-ordered. The TGE calculation must be performed using the state before the current purchase is factored in.

Resolution: In commit `a893459`, the TGE calculation logic is re-ordered, the new implementation performs calculation before the current purchase is factored in.

[F-2025-11742](#) - Incorrect Handling of reserveFee During dex() Execution - Medium

Description:

The `BondingCurve` contract defines a fee structure for the project token that includes several components like `reserveFee`, `teamFee`, `ecosystemFee`, `tokenToSellPercent`, and `LPPercent`. These fees are intended to be distributed to their respective beneficiaries when the `dex()` function is called, with the exception of `tokenToSellPercent`, which follows a vesting schedule.

According to the contract's logic and documentation, the `dex()` function should handle the full distribution of fees, including the `reserveFee`. However, during execution, while most fee components are transferred correctly, the `reserveFee` is not transferred and instead remains locked in the bonding curve contract.

```
function dex() public onlyRole(DEFAULT_ADMIN_ROLE) {
    ...
    IRouter_ETH_BNB_AVAX(router).addLiquidity(
        ...
    )
    _giveToken(bondedToken, setUp.platformOwner, bondedTaxAmount);
    _giveToken(bondedToken, setUp.projectOwner, remainingAmount);

    if (curatorAmount > 0) {
        _giveToken(bondedToken, setUp.curatorAddress, curatorAmount);
    }

    _giveToken(projectToken, setUp.teamWallet, ledger.teamAmount);
    _giveToken(projectToken, setUp.ecosystemWallet, ledger.ecosystemAmount);
    uint256 remainingProjectToken = ledger.tokenToSell -
        ledger.totalProjectTokenSold;
    if (remainingProjectToken > 0) {
        _giveToken(projectToken, setUp.projectOwner, remainingProjectToken);
    }
}
```

This incomplete fee distribution results in a deviation from the intended economic model, leaving the `reserveFee` unclaimed and potentially disrupting the reserve's role in maintaining the curve's liquidity or price stability. It may also cause accounting inconsistencies and erode trust in the automated mechanisms of the token distribution logic for the potential users.

Assets:

- BondingCurveWritableRestricted.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:**Fixed**

Classification**Impact Rate:** 3/5**Likelihood Rate:** 5/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** **Medium**

Recommendations**Remediation:**

Consider transferring the `reserveFee` to the designated beneficiary within the execution of the `dex()` function to ensure full and accurate distribution of fees as intended by the contract's logic and documentation.

Resolution:

Fixed in `eb0e63e`: `reservedFee` is transferring to the `platformOwner` address once `dex()` function is called.

[F-2025-11752](#) - Incorrect Fee Math Allows Users to Exceed Allocation - Medium

Description:

The `BondingCurveWritable` contract facilitates a multi-tiered token sale where different user groups (Tiers, Whitelist, Public Sale.) are given specific purchase allocations. Participation in these private rounds is managed by a Merkle tree, where each leaf contains a user's address, their maximum allocation, and their designated round. When a user calls the `buy` function, the `_authenticateUser` internal function is invoked to verify their Merkle proof and check if their cumulative purchase amount is within their specified `allocation.amount`. This check is critical for enforcing the sale rules and ensuring a fair distribution.

The mathematical formula used within `_authenticateUser` to calculate a user's cumulative spend is incorrect. It attempts to reconstruct the user's total gross (pre-fee) contribution by applying a multiplicative fee factor to their stored net (post-fee) contribution, which underestimates the true amount spent. This flaw allows any user in a private round to systematically purchase more tokens than their official allocation without triggering the `ExceedAllocation` error, thereby bypassing a core security mechanism of the sale.

When a user makes a purchase, the contract stores their contribution after deducting a `preSwapFee`. This stored value is `user.bondedTokenAmount`. To check against the allocation on a subsequent purchase, the contract must reconstruct the user's total gross spend.

```
if (
    (strg.user[msg.sender].bondedTokenAmount) *
        ((10000 + strg.setUp.bondedTokenDetails.preSwapFee) / 10000) +
        amount >
    allocation.amount
) {
    revert ExceedAllocation();
}
```

This logic can be simplified to: `previous_net_spent * (1 + fee_rate) + current_gross_spend > allocation`.

This formula is mathematically flawed. The correct way to derive a gross amount from a net amount is `gross = net / (1 - fee_rate)`. The contract's formula `net * (1 + fee_rate)` will always result in a value lower than the actual gross amount. Because the check is performed against an artificially low value, it creates a loophole where

`previous_gross_spent + current_gross_spent` can exceed `allocation.amount` while the check still passes.

This allows every user in a private round to bypass their cap and purchase slightly more tokens than they are entitled to.

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: It is strongly recommended to refactor the allocation tracking mechanism to be based on gross amounts, which avoids error-prone reconstructions and precision loss from division.

Resolution: In commit `5529582`, the allocation tracking mechanism is implemented based on gross amounts.

Evidences

PoC

Reproduce:

1. Move the sale to an active private round (Tier 9) and obtain the user's valid Merkle proof for a 200-token allocation cap.
2. The user makes an initial purchase of 199.9 tokens (gross).
3. After the 0.75 % pre-swap fee is deducted, the contract stores only the net amount in `user.bondedTokenAmount`. The user immediately makes a second, tiny purchase of 0.11 tokens (gross).

4. `_authenticateUser` tries to reconstruct the previous gross spend with its faulty formula `net * (1 + feeRate)`; this under-estimates the real 199.9 tokens already spent, so the allocation check passes.
5. Both buys succeed, meaning the user's cumulative gross spend is now 200.01 tokens—already above their 200-token allocation—yet no `ExceedAllocation` revert is triggered.

PoC Code:

```
import { loadFixture, time } from "@nomicfoundation/hardhat-toolbox-viem/network-helpers";
import { expect } from "chai";
import { parseEther } from "viem";
import { BondingCurveSetupFixture } from "../BondingCurveSetup.fixture";

describe("BondingCurve – Allocation bypass due to fee math", function () {
  it("User can overspend their allocation by a small amount without revert", async function () {
    const { accounts, setUp, bondingCurve, paymentToken, getProof } = await loadFixture(
      BondingCurveSetupFixture
    );

    await time.increaseTo(Number(setUp.bondingCurveTimestamps.tier9));
    const proof = (await getProof(accounts.user1.address)) as `0x${string}`[];

    const allocationGross = parseEther("200");

    const firstBuy = parseEther("199.9");
    const secondBuy = parseEther("0.11");

    //First purchase
    await bondingCurve.write.buy(
      [
        {
          user: accounts.user1.address,
          amount: allocationGross,
```

```

roundType: 0,
},
proof,
firstBuy,
0n,
],
{ account: accounts.user1 }
);

//Second purchase (should exceed allocation)
await bondingCurve.write.buy(
[
{
user: accounts.user1.address,
amount: allocationGross,
roundType: 0,
},
proof,
secondBuy,
0n,
],
{ account: accounts.user1 }
);

const user = await bondingCurve.read.getUser([accounts.user1.address]);

const feeBps = BigInt(setUp.bondedTokenDetails.preSwapFee);
const grossSpent =
(user.bondedTokenAmount * 10000n) / (10000n - feeBps);

expect(grossSpent).to.be.greaterThan(allocationGross);
});
});

```

Results:

```

BondingCurve – Allocation bypass due to fee math
ending Price: 2000000000000000000n
totalSupply: 58000000000000000000000000000000n
ending Price: 20000000000000000000000000000000n
HARD CAP: 836763123715598776184282919n
hardCapIncludeTaxAmount: 843086270746195240487942487n
✓ User can overspend their allocation by a small amount without revert (2

```

915ms)

[F-2025-11767](#) - Inconsistent State Change in autoRefund() Affects TGE Unlock and Price Logic - Medium

Description:

The `BondingCurve` contract provides a mechanism for the default admin to refund users via `autoRefund()` function if the `softCap` has not been reached. The refund process is expected to return the user's bonded tokens, remove their project token claim, and return that token amount to the available pool so other users can purchase them.

The bonding curve maintains two key state variables:

- `tokenToSell`: the fixed amount of tokens intended to be sold during the curve lifecycle, calculated once on deployment.

```
function _updateLedger(  
    ProjectTokenDetails calldata projectTokenDetails,  
    Ledger storage ledger,  
    uint256 tSupply  
) internal {  
    ledger.tokenToSell =  
        (tSupply * projectTokenDetails.tokenToSellPercent) /  
        10000;  
    ...  
}
```

- `totalProjectTokenSold`: the amount of tokens sold so far, which should decrease if tokens are refunded.

In the `_buy()` function, the contract checks whether the remaining tokens (`tokenToSell - totalProjectTokenSold`) are sufficient for the new purchase.

However, the `autoRefund()` function currently adds the refunded tokens back to `tokenToSell`, instead of subtracting them from `totalProjectTokenSold`. This behavior deviates from the expected logic, where `tokenToSell` should remain constant, and only `totalProjectTokenSold` should be updated.

```
function autoRefund(  
    address[] calldata userAddresses  
) external whenNotPaused onlyRole(DEFAULT_ADMIN_ROLE) {  
    if (userAddresses.length == 0) revert EmptyArray();  
  
    BondingCurveStorage.Storage storage strg = BondingCurveStorage.layout();  
    if (strg.ledger.raisedTokenAmount >= strg.setUp.softCap)
```



```

        revert RefundNotAllowed();

        address token = strg.setUp.bondedTokenDetails.token;
        uint256 totalBondedAmount;
        uint256 totalClaimAmount;

        for (uint256 i; i < userAddresses.length; i++) {
            User storage user = strg.user[userAddresses[i]];
            uint256 bondedAmount = user.bondedTokenAmount;
            if (bondedAmount > 0) {
                totalBondedAmount += bondedAmount;
                totalClaimAmount += user.amountToClaim;

                user.bondedTokenAmount = 0;
                user.amountClaimed = type(uint256).max;

                _giveToken(token, userAddresses[i], bondedAmount);
                emit Refunded(
                    userAddresses[i],
                    strg.ledger.raisedTokenAmount,
                    getAmountOutForBondedToken(1 ether)
                );
            }
        }

        strg.ledger.tokenToSell += totalClaimAmount; // <-- incorrect state change
        strg.ledger.raisedTokenAmount -= totalBondedAmount;
    }
}

```

This incorrect state change affects multiple contract logic aspects:

- **Project token sale cap misrepresentation:** `tokenToSell` no longer accurately reflects the intended total supply for sale.
- **Pricing errors:** The price per token, calculated in the `_area0()` function using the ratio `sWei / supplyTokens`, will be slightly higher than it should be due to an inflated `supplyTokens` (represent `tokenToSell`) value.

```

function _area0(
    uint256 sWei,
    uint256 endingPrice,
    uint256 multiplierHuman,
    uint256 sellableSupply,
    int128 lnM64x64
) private pure returns (uint256) {
    // number of tokens (human)
    uint256 supplyTokens = sellableSupply / SCALE;
}

```

```

// compute b^(s/S) in Q64.64
uint128 fracQ64 = (sWei / SCALE).fromUInt().div(supplyTokens.fromUInt());
// <-- outdated ratio
...
}

```

- **Incorrect TGE unlock amount:** The TGE amount becomes skewed, causing users to receive a lower initial unlock than intended.

```

function _calculateTGEPercent(
    uint256 totalTokenToSell,
    uint256 totalProjectTokenSold,
    uint256 baseTGEUnlockPercent
) internal pure returns (uint256) {
    uint256 ratio = (totalProjectTokenSold * 1e18) / totalTokenToSell; // <--
    outdated ratio
    uint256 scaledUnlock = baseTGEUnlockPercent *
        1e18 +
        ratio *
        (10000 - baseTGEUnlockPercent);

    return scaledUnlock / 1e18;
}

```

- **Broken inverse curve calculations:** In `_inverseArea0()`, the `sellableSupply` is derived from `tokenToSell`, and an inflated value leads to incorrect area calculations, impacting expected token supply and price behavior.

```

function _inverseArea0(
    uint256 targetArea,
    uint256 endingPrice,
    uint256 multiplierHuman,
    uint256 sellableSupply,
    int128 lnM64x64
) private pure returns (uint256) {
    uint256 supplyTokens = sellableSupply / SCALE;

    // A = endingPrice * supplyTokens * MULTIPLIER_SCALE / multiplierHuman
    uint256 A = (endingPrice * supplyTokens * MULTIPLIER_SCALE) / multiplierHuman;
    <-- outdated
    ...
}

```

Meanwhile, the `refund()` function, which allows users to perform individual refunds under the same conditions, updates the state correctly: only `totalProjectTokenSold` is modified, while `tokenToSell` remains unchanged.

```
function refund() external whenNotPaused {
    ...

    uint256 bondedAmount = user.bondedTokenAmount;
    uint256 claimAmount = user.amountToClaim;

    user.bondedTokenAmount = 0;
    user.amountClaimed = type(uint256).max;

    strg.ledger.raisedTokenAmount -= bondedAmount;
    strg.ledger.totalProjectTokenSold -= claimAmount; // <-- correct state change
    user.isRefunded = true;
    ...
}
```

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	4/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Consider updating the `autoRefund()` function to correctly subtract the refunded project tokens from `totalProjectTokenSold` instead of adding them to `tokenToSell`. This will preserve the integrity of the bonding curve's economics, ensure accurate price and initial TGE unlock calculations, and maintain consistency with the curve's intended design.

Resolution:

Fixed in `c4b53ea`: the value of `totalProjectTokenSold` is now correctly updated by subtracting `totalClaimAmount` of the refunded users.

[F-2025-11734](#) - Missing Validation on Critical Vesting Parameters - Low

Description:

The `initialize` function sets up the bonding curve with parameters provided in the `SetUp` struct, including the `VestingDetails`. The `_checkVestingSetup` internal function is intended to validate these parameters to ensure the contract operates correctly.

The contract fails to perform validation on the `baseTGEUnlockPercent` parameter. A deployer can provide a value greater than 100% (`10000`), which will cause the `_calculateTGEPercent` function to underflow and revert on the very first `buy` transaction. This will permanently brick the contract, making it impossible for anyone to purchase tokens and resulting in a Denial of Service.

The `_checkVestingSetup` function contains no validation for `vestingDetails.baseTGEUnlockPercent`.

```
function _checkVestingSetup(
    VestingDetails calldata vestingDetails,
    bytes32 merkleRoot,
    BondingCurveType bondingCurveType
) internal pure {
    if (
        merkleRoot == bytes32(0) &&
        bondingCurveType == BondingCurveType.Curator
    ) {
        revert InvalidMerkleRoot();
    }
    if (
        vestingDetails.endDate <
        vestingDetails.startDate + vestingDetails.cliff
    ) {
        revert InvalidVestingTimings();
    }
}
```

If `baseTGEUnlockPercent` is set to a value greater than 10000, the calculation within `_calculateTGEPercent` will underflow. Since the `initialize` function is only called once and its parameters are immutable, a mistake here is irreversible. The first user attempting to `buy` will trigger the underflow, causing their transaction to revert. All subsequent `buy` calls will also fail, rendering the bonding curve unusable.

An invalid `baseTGEUnlockPercent` will make all `buy` and `buyPublic` transactions permanently fail.

Assets:

- BondingCurveWritableRestricted.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Add a strict validation check in `_checkVestingSetup` to ensure that `baseTGEUnlockPercent` is less than or equal to `10000`.

Resolution: In commit `a667923`, the following validation is added into the `_checkVestingSetup` function:

```
if (vestingDetails.baseTGEUnlockPercent > 10000) {  
    revert InvalidPercent();  
}
```

[F-2025-11748](#) - Incorrect Calculation Can Bypass Vesting

Schedule - Low

Description:

The vesting schedule is governed by the `startDate`, `cliff`, and `endDate` fields within the `VestingDetails` struct. This schedule is critical for ensuring a controlled release of tokens into the market, which protects investors from the price shocks of a sudden supply increase and demonstrates long-term commitment from the project team.

The contract's validation logic for vesting timings is insufficient. It is possible for a deployer to configure the vesting parameters in such a way that the entire linear vesting period is bypassed. By setting the `cliff` to zero and the `endDate` equal to the `startDate`, 100% of a user's purchased tokens (both TGE and vested portions) become immediately claimable at the vesting `startDate`, completely defeating the purpose of a vesting schedule.

The `_getClaimableAmount` function determines the claimable amount based on the current time relative to the vesting schedule. If the `block.timestamp` is greater than or equal to `vestingDetails.endDate`, the function allows the user to claim their total `amountToClaim`.

```
function _getClaimableAmount(
    User memory user,
    VestingDetails memory vestingDetails
) internal view returns (uint256 claimableAmount) {
    uint256 amount = user.amountToClaim;
    if (block.timestamp < vestingDetails.endDate) {
        claimableAmount = _claimableAmount(
            vestingDetails,
            amount,
            startAmount,
            1e36
        );
    } else {
        claimableAmount = amount;
    }
    claimableAmount -= user.amountClaimed;
}
```

The validation in `_checkVestingSetup` only checks that `endDate` is not less than `startDate + cliff`.

```
if (
    vestingDetails.endDate <
```

```
vestingDetails.startDate + vestingDetails.cliff
) {
    revert InvalidVestingTimings();
}
```

This check does not prevent the case where `endDate == startDate` and `cliff == 0`. If a deployer sets these values, then as soon as `block.timestamp >= startDate`, the else block in `_getClaimableAmount` will be executed, making all tokens immediately available.

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: The contract must enforce a minimum vesting duration to prevent instant unlocks. The validation logic in `_checkVestingSetup` should be strengthened significantly.

Resolution: The client has acknowledged the finding with the following statement:

On some of the bonding curve vesting can be optional.

F-2025-11655 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.23`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- BondingCurve.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]
- BondingCurveLibrary.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]
- MyToken.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts>]
- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]
- BondingCurveWritableRestricted.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]
- BondingCurveReadable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]
- BondingCurveTypes.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates.

Resolution: In commit [1761ce3](#), all pragmatic versions are locked.

[F-2025-11743](#) - Unused Custom Errors and Events - Info

Description: There are unused custom error and events declarations within the codebase, including:

Custom errors:

- **BondingCurveTypes:** `InvalidBondingCurveTimings()` and `AlreadyClaimed()`
- **BondingCurveWritableRestricted:** `InvalidSignature()`
- **IBondingCurveFactoryInternal:**
 - `BondingCurveFactory_BondingCurve_ZeroAddr()`,
 - `BondingCurveFactory_BondingCurveIdExists()`,
 - `BondingCurveFactory_CurveAlreadyPaid()`, `AmountMustBeGreaterThanZero()`,
 - `InsufficientFunds()`, `BondingCurveFactory_CurveNotPaid()`,
 - `BondingCurveFactory_Token_ZeroAddr()`,
 - `BondingCurveFactory_Validator_ZeroAddr()`,
 - `BondingCurveFactory_OFTFactory_ZeroAddr()`

Events:

- **IBondingCurveFactoryInternal:** `FundsWithdrawn()` and `FeePaid()`

These custom errors and events are defined but never referenced, thrown or emitted anywhere in the current implementation of the contracts. Unused code contributes to unnecessary bytecode size, potentially increasing deployment and execution costs. It can also reduce code clarity and maintainability over time.

Assets:

- BondingCurveWritableRestricted.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]
- BondingCurveTypes.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity:

Info

Recommendations**Remediation:**

Consider removing unused custom errors and events to keep the codebase clean and optimized. Alternatively, if these errors or events are intended for use, ensure they are implemented in appropriate locations where they add meaningful validation.

Resolution:

Fixed in [721d574](#): all unused custom errors and events were removed.

[F-2025-11744](#) - Typographical Error in BondingCurveTypes - Info

Description:

In the `BondingCurveTypes` contract, the enum `BondingCurveRountType` contains a typographical error in its name. Specifically, `BondingCurveRountType` is misspelled and should be corrected to `BondingCurveRoundType`.

```
enum BondingCurveRountType { // <--  
    Tier9,  
    Tier8,  
    Tier7,  
    Tier6,  
    Tier5,  
    Tier4,  
    Tier3,  
    Tier2,  
    Tier1,  
    Whitelist,  
    kol,  
    publicSale  
}
```

Although this does not introduce a functional or security vulnerability, it is a code quality issue that should be addressed.

Assets:

- `BondingCurveTypes.sol` [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider renaming `BondingCurveRountType` to `BondingCurveRoundType` in the `BondingCurveTypes` abstract contract.

Resolution: Fixed in `b9d9f8c`: the `BondingCurveRountType` was renamed to `BondingCurveRoundType` in the `BondingCurveTypes` abstract contract.

[F-2025-11749](#) - Misleading 'start' Field in BondingCurveTimestamps Struct - Info

Description:

The `BondingCurve` contract allows users to purchase tokens during private and public rounds through the `buy()` and `buyPublic()` functions. These functions rely on time-based validation logic to determine whether a round is active.

The `BondingCurveTimestamps` struct defines multiple time-related fields, including individual round start times and a general `start` field meant to represent the start date of the entire bonding curve.

```
struct BondingCurveTimestamps {
    uint32 start;
    uint32 end;
    uint32 tier9;
    uint32 tier8;
    uint32 tier7;
    uint32 tier6;
    uint32 tier5;
    uint32 tier4;
    uint32 tier3;
    uint32 tier2;
    uint32 tier1;
    uint32 whitelist;
    uint32 kol;
    uint32 publicSale;
}
```

However, in the implementation of the buy functions, only the per-round start dates are used for validation. As a result, the `start` field remains unused.

Private round: `buy():_authenticateUser()`:

```
function _authenticateUser(
    BondingCurveStorage.Storage storage strg,
    uint256 amount,
    UserAllocation calldata allocation,
    bytes32[] calldata merkleProof
) internal view {
    uint256 roundStart = _getRoundStartDate(
        strg.setUp.bondingCurveTimestamps,
        allocation.roundType
    );
}
```

```

        if (block.timestamp < roundStart) {
            revert RoundNotStarted();
        }
        ...
    }
}

```

Public round: `buyPublic()`:

```

function buyPublic(
    uint256 amount,
    uint256 minAmount
) external whenNotPaused {
    BondingCurveStorage.Storage storage strg = BondingCurveStorage.layout
();
    if (block.timestamp < strg.setUp.bondingCurveTimestamps.publicSale) {
        revert RoundNotStarted();
    }
    _buy(strg, amount, minAmount);
}

```

The presence of unused struct fields increases bytecode size, which may lead to higher deployment and execution costs. It also reduces code clarity and may mislead developers about the structure's relevance or purpose.

Assets:

- BondingCurveTypes.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Consider removing the unused `start` field from the `BondingCurveTimestamps` struct to avoid confusion, improve code clarity,

and optimize the contract's bytecode size.

Resolution:

Fixed in `f7e58d1`: the `start` field was removed from the `BondingCurveTimestamps` struct.

[F-2025-11750](#) - Redundant SafeERC20 and IERC20 Imports

Declaration - Info

Description: The `BondingCurveFactory` contract is responsible for creating and tracking bonding curves. The `SafeERC20` library is imported and declared for use with the `IERC20` type within the contract.

```
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract BondingCurveFactory is
    IBondingCurveFactory,
    AccessControlEnumerable,
    ReentrancyGuard,
    IBondingCurveFactoryInternal
{
    using SafeERC20 for IERC20;
    ...
}
```

Despite the declaration, none of the `SafeERC20` functions are actually used within the contract. Additionally, the `BondingCurveFactory` does not perform any ERC20-related operations that would require the safety wrappers provided by the library.

The same redundant imports appears in the `BondingCurveWritable` contract. The `SafeERC20Upgradeable` library is imported and declared for use with the `IERC20Upgradeable` type within the contract. Same as `BondingCurveFactory` the `BondingCurveWritable` contract does not perform any ERC20-related operations.

```
import {SafeERC20Upgradeable} from "@openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol";
import {IERC20Upgradeable} from "@openzeppelin/contracts-upgradeable/token/ERC20/IERC20Upgradeable.sol";

contract BondingCurveWritable is
    IBondingCurveWritable,
    BondingCurveWritableRestricted
{
    using SafeERC20Upgradeable for IERC20Upgradeable;
    ...
}
```

As a result, a redundant library unnecessarily increases the contract size and compiled bytecode. While the impact may be minor, it contributes to reduced optimization and could be misleading to users.

Assets:

- BondingCurveFactory.sol [<https://github.com/Seedifyfund/seedify-era-contracts/blob/develop/contracts/factory>]
- BondingCurveWritable.sol [<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:**Fixed**

Classification**Impact Rate:** 1/5**Likelihood Rate:** 3/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Info

Recommendations

Remediation: Consider removing the redundant `SafeERC20`, `SafeERC20Upgradeable` library and `IERC20`, `IERC20Upgradeable` interface declarations from `BondingCurveFactory` and `BondingCurveWritable` contracts to reduce bytecode size and keep the codebase clean and maintainable.

Resolution: Fixed in `3f42cb2`: redundant `SafeERC20`, `SafeERC20Upgradeable` library and `IERC20`, `IERC20Upgradeable` interface declarations were removed from `BondingCurveFactory` and `BondingCurveWritable`.

[F-2025-11751](#) - State Change of `_maxLoop` Without Event in BondingCurveFactory Contract - Info

Description: The `BondingCurveFactory` contract includes a mutable state variable `_maxLoop`, which determines the maximum number of items that can be processed by the `getBondingCurvesDetails()` function. This variable can be updated via the `setMaxLoop()` function.

```
function setMaxLoop(
    uint256 newMaxLoop
) external override onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newMaxLoop == 0) revert BondingCurveFactory_MaxLoop_Zero();
    _maxLoop = newMaxLoop;
}
```

Although `_maxLoop` directly affects how much data can be retrieved in one call, no event is emitted when `setMaxLoop()` is called.

Without an event emission, changes to `_maxLoop` are not easily trackable by off-chain services such as indexers, front-end applications, or monitoring tools.

Assets:

- `BondingCurveFactory.sol` [<https://github.com/Seedifyfund/seedify-era-contracts/blob/develop/contracts/factory>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider emitting an event in the `setMaxLoop()` function to address the `_maxLoop` variable state changes. This would enhance transparency,

simplify monitoring, and align the contract with best practices for observable state changes.

Resolution:

Fixed in `38f3e7b`: new event was introduced `maxLoopUpdated()` and emitted in the `setMaxLoop()` function.

[F-2025-11759](#) - Redundant OFTTypes and BondingCurveStorage Import Declaration - Info

Description: `BondingCurveFactory` contract inherits from the `IBondingCurveFactory` and `IBondingCurveFactoryInternal` interfaces. Both interfaces contain redundant import declaration that never used within the codebase:

- **IBondingCurveFactory:** `OFTTypes`
- **IBondingCurveFactoryInternal:** `BondingCurveStorage`

Redundant libraries unnecessarily increases the contract size and compiled bytecode. While the impact may be minor, it contributes to reduced optimization and could be misleading to users.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing unused import statements of `OFTTypes` and `BondingCurveStorage` from the interfaces to keep the codebase clean, optimized, and easier to maintain.

Resolution: Fixed in `698fe98`, unused imports of `OFTTypes` and `BondingCurveStorage` were removed.

[F-2025-11765](#) - Unsafe Salt Generation with abi.encodePacked - Info

Description:

`_createBondingCurve`, is responsible for deploying new bonding curve instances. To ensure that each created curve has a predictable and unique address, it utilizes the `Clones.cloneDeterministic` function. This function generates a new contract at an address derived from a combination of the factory's address, the implementation address, and a unique salt. The contract attempts to create this salt by hashing the `_msgSender` and a user-provided `curveName`.

The contract generates the salt for deterministic deployment using `abi.encodePacked` with dynamic data types (`string`). This method is susceptible to hash collisions, where different inputs can produce the same output hash. An attacker could exploit this by crafting a specific `curveName` that collides with an existing one, leading to a Denial of Service (DoS) condition that would prevent new bonding curves from being created under certain names.

```
bytes32 salt = keccak256(abi.encodePacked(_msgSender(), curveName));
address curve = Clones.cloneDeterministic(_defaultBondingCurve, salt);
```

The `abi.encodePacked` function concatenates arguments without padding, which can create ambiguities when used with dynamic types like bytes or string. For example, `abi.encodePacked("attack", "one")` and `abi.encodePacked("attackone")` will produce the same output if the second argument is an empty string in the first case.

In this context, a malicious actor (or even an admin by mistake) could provide two different `curveName` strings that result in the same packed encoding, and therefore the same salt. While the contract checks if a `curveName` has already been used with the line `if (_curveDetail[curveName].bondingCurve != address(0))`, this check does not prevent a hash collision from two different `curveNames`. If a collision occurs for the same `_msgSender`, the `Clones.cloneDeterministic` call will attempt to deploy a new contract to an address that is already occupied by a previously deployed curve, causing the transaction to revert.

Assets:

- BondingCurveFactory.sol [<https://github.com/Seedifyfund/seedify-era-contracts/blob/develop/contracts/factory>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	To ensure that the salt generation is safe and unambiguous, <code>abi.encodePacked</code> should be replaced with <code>abi.encode</code> .
Resolution:	In commit <code>f971440</code> , <code>abi.encode</code> is adopted instead of <code>abi.encodePacked</code> .

F-2025-11948 - Code Contains `hardhat` Import - Info

Description:

In Solidity development, `hardhat` is a commonly used development framework that provides tools for testing, debugging, and deploying smart contracts. However, imports from `hardhat` in your Solidity code, such as those used for testing or local development, should not make their way into the final production version of the contract. These imports can bloat the contract, lead to unnecessary complexity, and potentially introduce security risks or unexpected behavior. Ensuring that production code is clean, efficient, and free of development-only dependencies is crucial for maintaining security and performance.

```
import "hardhat/console.sol";
```

Assets:

- BondingCurveWritable.sol
[<https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Before finalizing and deploying your Solidity contracts, thoroughly review the code to ensure all `hardhat` imports and related development-only code are removed.

Resolution:

In commit `3c23375`, the `hardhat` import is removed.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Seedifyfund/seedify-era-contracts
Initial Commit	7b054d9
Final Commit	1761ce3
Whitepaper	N/A
Requirements	Provided as file
Technical Requirements	Provided as file

Asset	Type
BondingCurve.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveFactory.sol [https://github.com/Seedifyfund/seedify-era-contracts/blob/develop/contracts/factory]	Smart Contract
BondingCurveLibrary.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveReadable.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveStorage.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveTypes.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveWritable.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
BondingCurveWritableRestricted.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts/bondingcurve]	Smart Contract
MyToken.sol [https://github.com/Seedifyfund/seedify-era-contracts/tree/develop/contracts]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of Seedify.fund, Hacken followed its methodology by performing fuzz-testing on the project's main functions. [Foundry](#), a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, **10** invariants were tested over **100,000** runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
<code>totalProjectTokenSold</code> cannot exceed <code>tokenToSell</code>	Passed	10K+
<code>amountClaimed</code> cannot exceed <code>amountToClaim</code>	Passed	10K+
<code>raisedTokenAmount</code> cannot exceed <code>hardCap</code>	Passed	10K+
<code>tgeAmount</code> cannot exceed <code>amountToClaim</code>	Passed	10K+
Should claim initial claimable TGE amount on cliff period	Passed	10K+
Should refund only if <code>softCap</code> not reached	Passed	10K+
Should add liquidity when <code>softCap</code> is reached only if curve is end	Passed	10K+
Should add liquidity once <code>hardCap</code> is reached	Passed	10K+
Cost per token cannot exceed <code>endingPrice</code>	Passed	10K+
Should only create curves with unique names	Passed	10K+

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.